

COMP2041

Week 3

me forgetting to chmod +x before running a shell script



BUGS

The man page is longer than the program.

spend
5 minutes
doing a
simple task

spend
half an hour
automating
a simple task

**Thank you Kyu-Sang Kim for the
foundation of these slides!**

- **Welcome to Week 3 of COMP2041**
- Lab 1 and 2 due two days ago (Monday 12PM AEST)
 - Lab 3 due Week 4 Monday 12PM AEST (midday)
- Week 03 Weekly Test available soon
 - Watch the lecture for more details
 - Taken under exam conditions
 - AFAIK one attempt, one hour but confirm at Lecture
- Help sessions are now being ran! Will show a tutorial on how to use them.

Overview

- Shell Scripting
- Tutorial Questions



Shell Scripting

When people ask how I learned to shell script



Shell Scripting

- In weeks 1 and 2, we have learnt how to use various command line tools
- Shell scripting allows us to automate the usage of command line tools with additional logic
- If you don't find at least one shell script somewhere at work, you need to move companies

Files as slides

- This tutorial I'll be interacting with files a lot more, and using them to replace slides. The files will have comments to explain code blocks
 - It's much easier in this context since I don't have to switch between slides too often, and I can interact with the code and change things to justify points.
- Please ask as much questions as you can, and I will be sure to add any additional clarifications to the notes in the files.
- I'll be sure to add these files and notes to the slides before I email them out.
- Let me know after the class if you also want access to the files.

Variable Basics

- Assign variables by doing **name=value** - no spaces around =.
- Access with \$name, or \${name}.
 - \${name} can be used within strings. -> "My name is \${name}".
- When evaluating a variable, the shell looks for the longest valid variable after '\$'
 - **echo \$xx**, looks for the variable 'xx', and prints the value inside
 - **echo \${x}x**, looks for the variable 'x', and prints the value inside, then prints the literal 'x'.

Variable Basics (cont.)

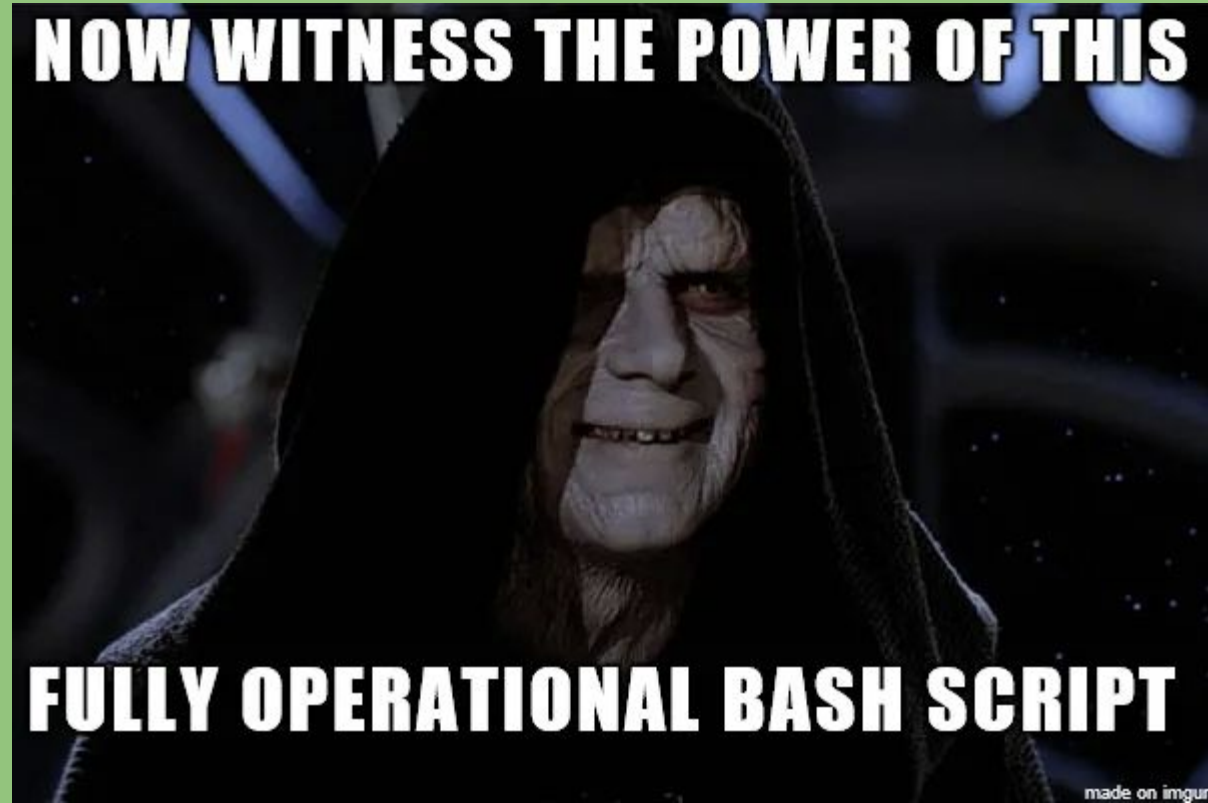
- **There is a key difference between single and double quotes**
 - Assume **x=5**
 - **echo "\$x" -> 5**
 - **echo '\$x' -> \$x**
 - Single quotes is completely literal.
- **Command substitution** allows you to execute commands in variables
 - **\$(...)** will run the command inside the parenthesis, and replaces itself with the output
 - Assume **z=ls**
 - **echo \$(z)** turns into **echo \$(ls)**, thus the output of **echo \$(z)** would just be the output of **ls**.

Word splitting

- By default, the shell will split expanded strings based on IFS (Internal Field Separator) which is all amounts of whitespace.
- Each chunk becomes a separated argument, and will also delete any extra spaces.
- **echo a b c -> a b c.** - deletes the extra spaces between **a** and **b**.
 - This is solved by using quotes. `echo "a b c" -> a b c`
- A variable containing spaces will be split into multiple arguments when unquoted.

If none of these makes sense now, it'll make sense by Q2 (hopefully)

Tutorial Questions



I will try my very best to not go over-time.

To The
Lab!

